We're using the --skip-test option to tell Rails not to use its default of Minitest. That's because in Chapter 7, we use RSpec to demonstrate how to configure our tests in a Dockerized environment.

We also use the --skip-bundle option. This tells Rails not to run bundle install after generating the project. The container is just a temporary vehicle for us to generate the Rails project—since we're going to get rid of it, there's no need to install the project dependencies.

When we run our rails new command, we get the following output, which shows our Rails project files being created, just as we'd expect:

```
create
create  README.md
create  Rakefile
create  .ruby-version
create  config.ru
create  .gitignore
create  Gemfile
«...»
create  vendor
create  vendor/.keep
create  storage
create  storage/.keep
create  tmp/storage
create  tmp/storage/.keep
remove  config/initializers/cors.rb
remove  config/initializers/new_framework_defaults_5_2.rb
```

Great! We can see our Rails files being generated. Remember, though, that we're *inside* the container, and we need to get the files onto our local machine. How do we do that?

First, let's terminate our Bash shell, which will stop the container:

```
root@0c286e8bda42:/usr/src/app# exit
```

This returns us to our familiar terminal prompt: $.

Now let's have a look inside the current directory on our local machine:

```
$ ls
myapp
$ cd myapp
$ ls
Gemfile     Rakefile   bin      config.ru   lib   package.json   storage
vendor      README.md  app      config      db    log            public
tmp
```